
Sistemas Operacionais II

Entrega 5

Anderson Simioni Assunção (23150566)

Bruno Pavani Bertolino (23100464)

Gustavo Fukushima de Salles (21250090)

Julia Fischer Gazolla (23250586)

Classes modificadas para P5

```
79 // ---- Steering ----
80 template <typename TNIC>
81 class SteeringComponent : public CarComponent<TNIC> {
82     using Base = CarComponent<TNIC>;
83     using Rx = typename Base::CommunicatorT::Rx;
84 public:
85     SteeringComponent(uint16_t port) : Base(port, "Steering") {}
86 protected:
87     bool wants_tick() override { return true; }
88     unsigned tick_period_ms() override { return 1000; }
89     void on_tick() override {
90         _angle = (_angle + 5) % 360;
91         std::string payload = "Steering angle " + std::to_string(_angle);
92         Base::send_local(payload);
93     }
94     void on_receive(const Rx& rx, ChannelOrigin origin) override {
95
96         if (rx.msg.body == "steer-left") _angle = (_angle + 350) % 360;
97         else if (rx.msg.body == "steer-right") _angle = (_angle + 10) % 360;
98         Base::send_local("ack-steering");
99     }
100 }
101 private:
102     int _angle{0};
103 };
104
```

- Novos componentes em components.h: steering, transmission e air bag.
- Cada novo componente segue tendo seu próprio arquivo de log

Melhorias gerais para P5

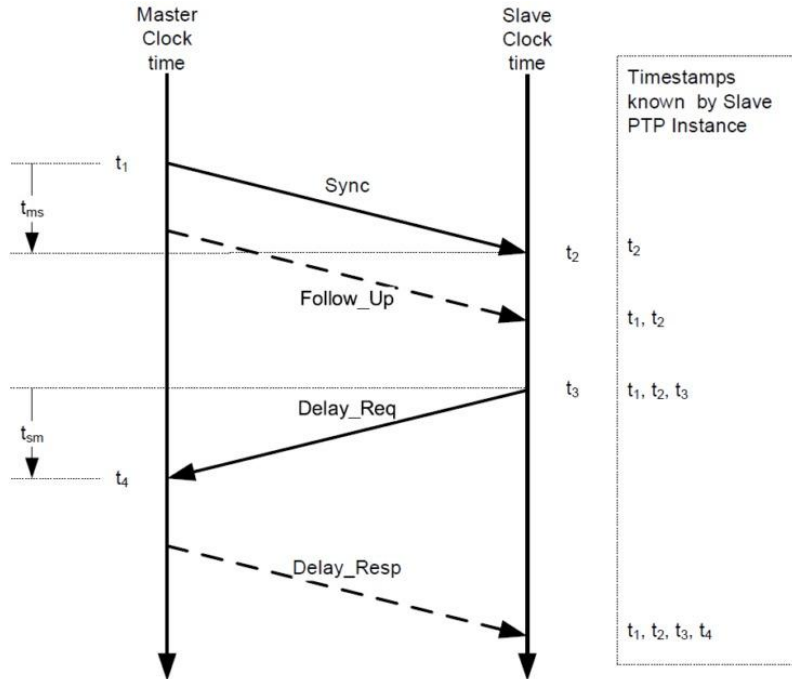
- Aprimorada estrutura Message para melhor encapsular os dados das mensagens, com método de build e parsing
 - Logs padronizados
 - Padronização da criação de componentes
 - Análise do tempo através de captura de pacotes via Wireshark
-

Arquitetura modificadas para P5

- Adicionamos uma nova VM, que simula equipamentos de medição na estrada
- Possui apenas o gateway como componente

```
components.h  car.cpp
c > car.cpp > Car()
10  Car::Car()
11
20
21  void Car::start(int vm_id, int total_sync_vms)
22  {
23      int components_len = this->components.size();
24
25      // Determine VM identity
26      bool is_sync_master = (vm_id == 0); // VM 0 is the road-side equipment (gateway only)
27
28      // Create the gateway for all VMs
29      Gateway<NIC>* gateway = new Gateway<NIC>(GATEWAY_PORT, vm_id, total_sync_vms, is_sync_master);
30      gateway->initialize(true, components_len);
31
32      // Only non-zero VMs (cars) fork child processes for other components
33      if(vm_id != 0)
34      {
35          for (int i = 0; i < components_len; i++)
36          {
37              int pid = fork();
38              if(pid == 0)
39              {
40                  // Child process: initialize component
41                  CarComponent<NIC>* comp = this->components.at(i);
42                  comp->initialize(false, components_len);
43                  return; // exit child after init
44              }
45          }
46      }
```

Elementos para Sincronização Temporal



- Modificação em **Protocol.h**: inclui separação das VMs por grupos e variável de controle para mensagens

```
69
70 // Message type: control or data
71 enum class ControlType {
72     NONE, // Not a control message
73     READY,
74     GO,
75     SYNC_REQ,
76     SYNC_RESP,
77     DELAY_REQ,
78     DELAY_RESP,
79     MOVE_GROUP,
80     GROUP_INFO,
81 };
82
```

Elementos para Sincronização Temporal

Protocol.h

```
344     else if(ctrl == ControlType::SYNC_REQ) {
345         if(!_is_master) {
346             // Master received SYNC_REQ from worker
347             uint64_t t1 = get_microseconds_now(); // master current time
348
349             char buf[128];
350             snprintf(buf, sizeof(buf), "%llu", (unsigned long long)t1);
351
352             //printf("[SYNC] SYNC_REQ received from %s, sending SYNC_RESP with t1=%llu\n", c.from.mac.str().c_str(), (unsigned
353
354             // Send SYNC_RESP with timestamp t1
355             send_control(_control_local, c.from, ControlType::SYNC_RESP, buf);
356         }
357     }
358
359     else if(ctrl == ControlType::SYNC_RESP && !_is_master) {
360         // Worker received SYNC from master
361         uint64_t t1 = 0;
362         if(!msg.body.empty()) sscanf(msg.body.c_str(), "%llu", (unsigned long long*)&t1);
363
364         uint64_t t2 = get_microseconds_now(); // worker receive time
365         uint64_t t3 = get_microseconds_now(); // worker send time (DELAY_REQ)
366
367         char buf[256];
368         snprintf(buf, sizeof(buf), "%llu %llu %llu", (unsigned long long)t1, (unsigned long long)t2, (unsigned long long)t3);
369
370         //printf("[SYNC] Received SYNC_RESP from %s (t1=%llu), sending DELAY_REQ (t2=%llu, t3=%llu)\n", c.from.mac.str().c_str(),
371             //(unsigned long long)t1, (unsigned long long)t2, (unsigned long long)t3);
372
373         send_control(_control_local, c.from, ControlType::DELAY_REQ, buf);
374     }
375 }
```

Elementos para Sincronização Temporal

Protocol.h

```
376     else if(ctrl == ControlType::DELAY_REQ && !_is_master) {
377         uint64_t t1, t2, t3;
378         if(sscanf(msg.body.c_str(), "%llu %llu %llu", (unsigned long long*)&t1, (unsigned long long*)&t2, (unsigned long long*)&t3) != 3) return;
379
380         uint64_t t4 = get_microseconds_now(); // master receive time
381
382         // Compute NTP-style offset and delay
383         int64_t offset = ((int64_t)(t2 - t1) + (int64_t)(t3 - t4)) / 2;
384         int64_t rtt = (t4 - t1) - (t3 - t2);
385
386         //printf("[SYNC] DELAY_REQ from %s (t1=%llu t2=%llu t3=%llu t4=%llu)\n offset=%lld microseconds rtt=%lld microseconds\n", c.from.mac.str().c_str(),
387             //((unsigned long long)t1, (unsigned long long)t2, (unsigned long long)t3, (unsigned long long)t4, (long long)offset, (long long)rtt);
388
389         // Send DELAY_RESP with offset & rtt
390         char buf[128];
391         snprintf(buf, sizeof(buf), "%lld %lld", (long long)offset, (long long)rtt);
392
393         send_control(_control_local, c.from, ControlType::DELAY_RESP, buf);
394     }
395
396     else if(ctrl == ControlType::DELAY_RESP && !_is_master) {
397         int64_t offset = 0;
398         int64_t delay = 0;
399
400         sscanf(msg.body.c_str(), "%lld %lld", (long long*)&offset, (long long*)&delay);
401
402         _clock_syncer->addPtpSample(offset, delay);
403
404         auto sync_over_time = this->get_probabilistic_ptp_timeout();
405         auto enough_samples = _clock_syncer->hasEnoughSamplesCI(100, 1.96); //+-50us with 95% confidence
406
407         if(enough_samples || sync_over_time)
408         {
409             _clock_syncer->applySync();
410             set_running_ptp(false);
411         }
412         else
413         {
414             this->send_control(_control_local, Endpoint(Ethernet::Address::BROADCAST(), _control_local.port), Protocol<TNIC>::ControlType::SYNC_REQ);
415         }
416     }
```

Elementos para Sincronização Temporal

Modificação em **Gateway.h** para solicitar sincronização temporal a cada 3 segundos

```
86 void on_tick() override {
87     uint64_t now_us = get_microseconds_now(); // wall-clock timestamp in microseconds
88
89     // SYNC REQUEST every 3 seconds
90     if (!_sync_master) {
91         if (now_us - _last_sync_request_us.load() >= 3'000'000ULL) { // 3 seconds
92
93             if(!_protocol->get_running_ptp())
94             {
95                 _protocol->set_probabilistic_ptp_timeout(true);
96             }
97             else
98             {
99                 _protocol->set_running_ptp(true);
100                 _protocol->set_probabilistic_ptp_timeout(false);
101                 _last_sync_request_us.store(now_us);
102
103                 Endpoint master_endpoint(Ethernet::Address::BROADCAST(), _local.port); // broadcasting for now, o
104                 printf("[SYNC][VM %d] Sending SYNC_REQ to master\n", _vm_id);
105                 _protocol->send_control(_local, master_endpoint, Protocol<TNIC>::ControlType::SYNC_REQ);
106             }
107         }
108     }
109 }
110
111 }
```


Elementos para Sincronização Temporal

```
3
4  /*
5   | ClockSyncer is responsible for calculate the better clock time
6   | based on probabilistic RTT algorithm,
7   | its a simple average based clock correction.
8   | its a small version of NTP.
9   */
0
1  You, 50 minutes ago | 1 author (You)
2  class clockSyncer {
3  private:
4      std::vector<long long> rtt_samples;
5      std::vector<long long> offset_samples;
6
7      //once define alg, its not possible to change!
8      //to define the alg just call the function addNtpSample or addPtpSample
9      //according to desired alg
10     int alg = -1; // -1 = not defined, 0 = NTP, 1 = PTP
11     long long avg_rtt = 0;
12     long long avg_offset = 0;
13
14     //average helper, just calculate the average of a generic long long collection..
15     > long long average(const std::vector<long long>& v) const {
16
17     You, 56 minutes ago • clock
```

Elementos para Sincronização Temporal

```
//add new time sample
//localSend = when sent the ping
//localRecv = when got the reply
//remoteTime = what time the other machine
void addNtpSample(long long localSend, long long localRecv, long long remoteTime) ...

//add PTP sample
//t1 = time when master sent the SYNC
//t2 = time when slave received the SYNC
//t3 = time when slave sent the DELAY_REQ
//t4 = time when master received the DELAY_REQ
//formula from IEEE 1588
void addPtpSample(long long t1, long long t2, long long t3, long long t4) { ...

void addPtpSample(long long offset, long long delay) {
    if(this->alg == 0) { std::perror("Error on add PTP sample, NTP already selected!!"); return; }
    this->alg = 1;

    //store info to calculate probabilistic value then
    addOffsetMeasurement(offset, delay);
}
```

Elementos para Sincronização Temporal

```
//Z-values for Normal Curve
//99.99% = 3.891
//99.00% = 2.575
//97.50% = 2.2414
//95.00% = 1.96
//maxWidthUs = max total width of the interval you accept, in us, ex: 2.0 means average  $\pm 1$  us
//build a confidence interval around the mean:  $\text{mean} \pm z * \text{SE}$  and look at the total width:  $2 * z * \text{SE}$ 
//RESUME: say if its have enough samples according to the selected Z..
bool hasEnoughSamplesCI(long long maxWidthUs, double confidenceZ = 3.891) const
{
    size_t n = offset_samples.size();
    if(n >= MAX_SAMPLES_IN_CLOCK_SYNCER) return true;
    if(n < 30) return false; //keep collecting, 30 because its magic on Prob & Stat..,
    //but then calculate the n for Z

    long long se = getOffsetStdError(); //standard error of the mean
    long long width = (long long)2 * se * confidenceZ; //total width of the CI

    //return if the interval is good enough
    return width <= maxWidthUs;
}
```

Elementos para Sincronização Temporal

```
bool adjust_time_gradually(long long offset_us) const {
    // 0 adjtimex espera o offset em nanossegundos (us * 1000)
    long long offset_ns = offset_us * 1000LL;

    struct timex tx {};

    tx.modes = ADJ_OFFSET; // Ajust offset
    tx.offset = offset_ns;

    int result = adjtimex(&tx);

    if (result == -1) {
        std::perror("adjtimex failed");
        return false;
    }

    return true;
}

// Try to set the system clock using our best offset guess
bool applySync() const {
    long long offset_us = getAverageOffset();

    // Gradually sinc
    int result = adjust_time_gradually(offset_us);

    if (result == false) // 'false' if adjust_time_gradually faile
    {
        return false;
    }

    //printf("[SYNC] system clock adjusted by offset %lld us\n", offset_us);
    return true;
}
```

Conclusões

- De forma geral, a equipe conseguiu concluir as tarefas propostas para a quinta entrega do projeto de Comunicação em Sistemas Autônomos Críticos.
 - Foi possível implementar a sincronização em tempo real.
 - Próximos passos: criar e tratar grupos diferentes com uma VM master em cada grupo e troca de informações entre grupos.
-